

HANA CDS Views

TRUSTED INFOTECH

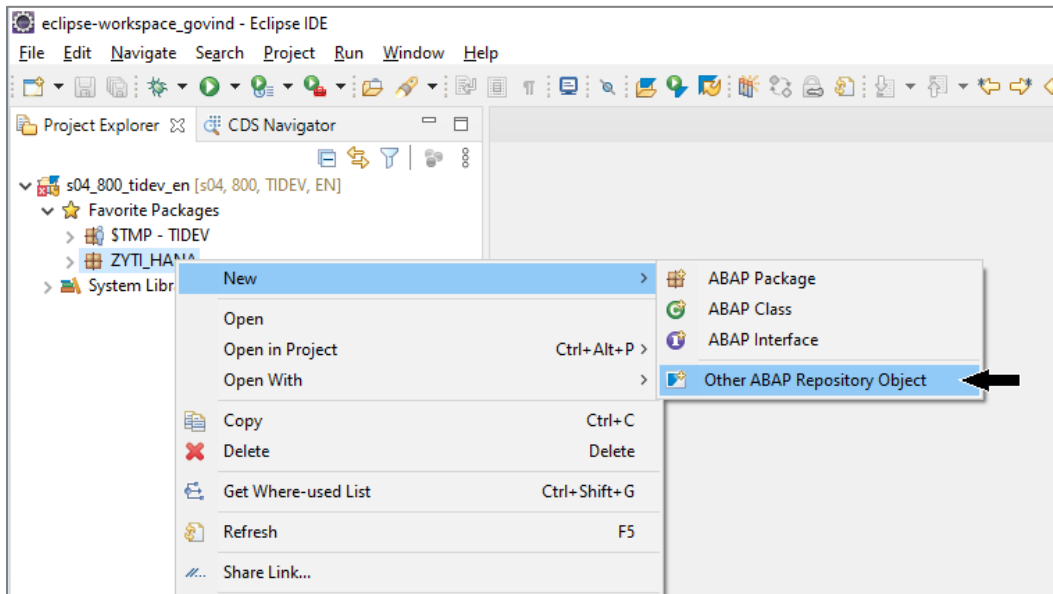
OCT 2020

CONTENT

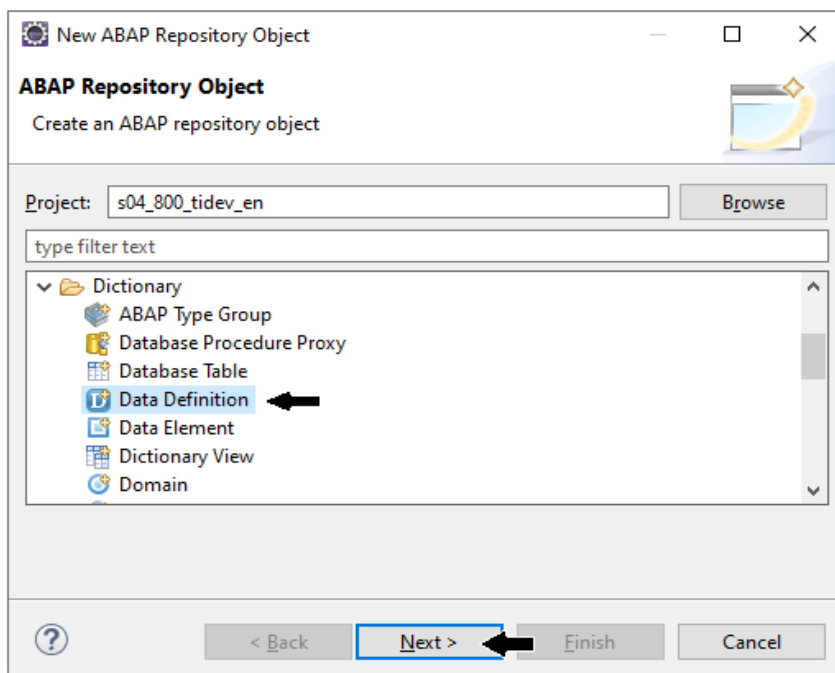
- 1. Creating CDS View**
- 2. Calling CDS Views in ABAP report**
- 3. JOINS in ABAP CDS Views on HANA**
 - 3.1 INNER JOIN or just JOIN
 - 3.2 LEFT OUTER JOIN
 - 3.3 RIGHT OUTER JOIN
- 4. CDS Views with Parameters**
 - 4.1 Parameters used in SELECT list
 - 4.2 Parameters used in Arithmetic Expressions
 - 4.3 Parameters used in CASE statement
- 5. Aggregate Expressions in ABAP CDS Views**
 - 5.1 SUM
 - 5.2 MAX
 - 5.3 MIN
 - 5.4 COUNT(*)
 - 5.5 COUNT(DISTINCT)
- 6. String Functions in ABAP CDS Views**
 - 6.1 CONCAT(arg1, arg2)
 - 6.2 CONCAT_WITH_SPACE(arg1, arg2, spaces)
 - 6.3 SUBSTRING(arg, pos, len)
 - 6.4 LENGTH(arg)
 - 6.5 LEFT(arg, len) & RIGHT(arg, len)
 - 6.6 LTRIM(arg, char) & RTRIM(arg, char)
- 7.CDS View Extensions**

1. Creating CDS View

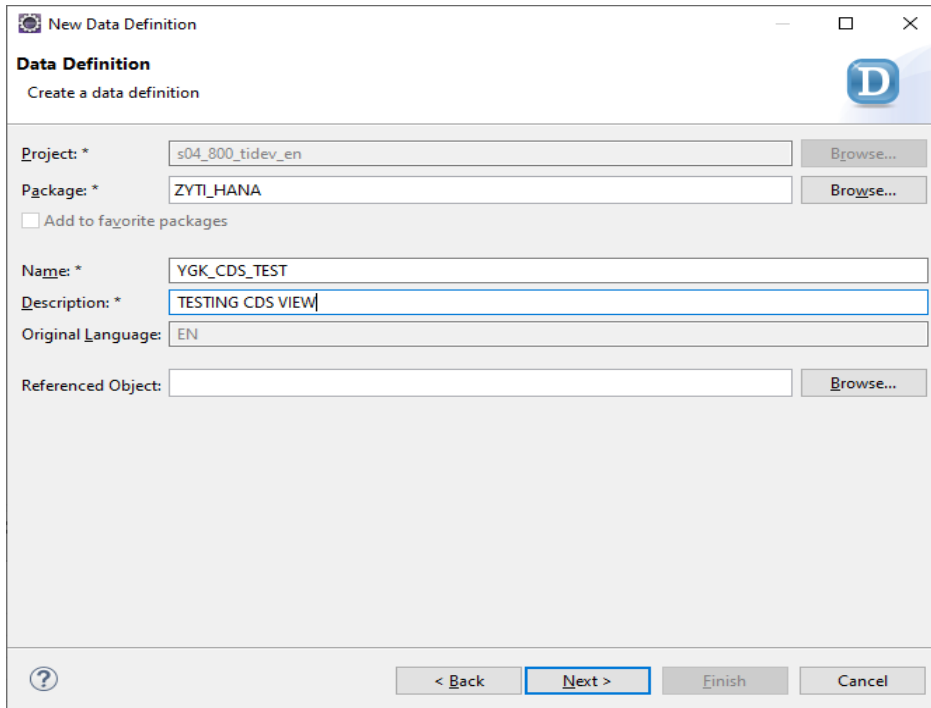
- 1. Choose the package in which you want to create CDS Views. Right-click on the package → **New** → **Other ABAP Repository Object**.



- 2. In the **New ABAP Repository Object** window, Open **Dictionary** folder and choose **Data Definition** and click on **Next**.



3. Enter the Name and Description of the CDS View and hit **Finish**.

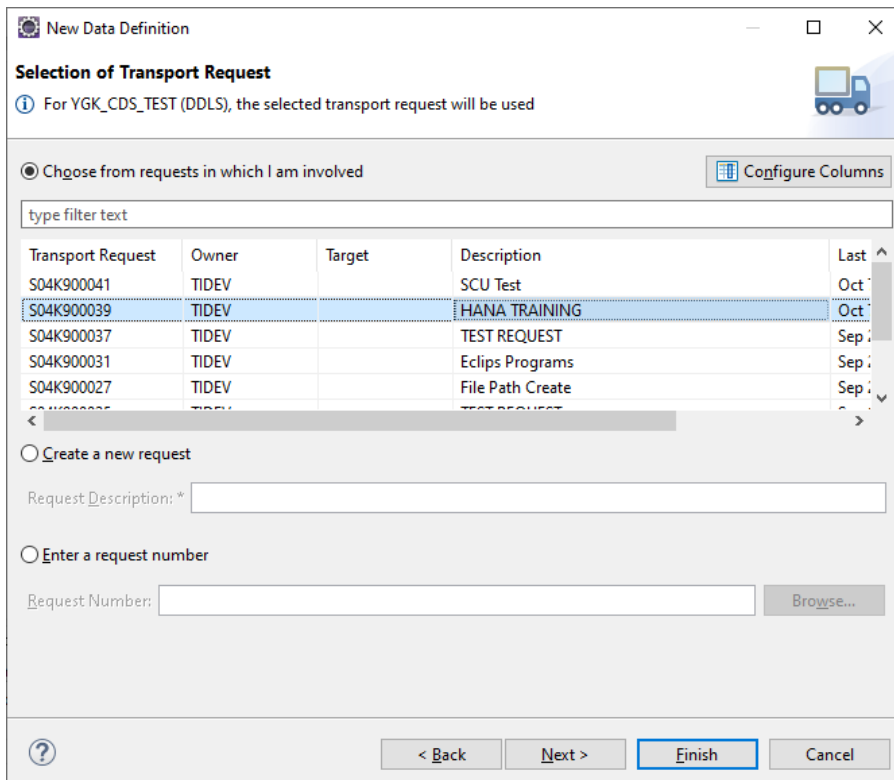


The 'New Data Definition' dialog box is shown. It has a title bar with a question mark icon, a close button, and a maximize button. The main title is 'Data Definition' with a subtitle 'Create a data definition'. There is a large blue 'D' icon in the top right corner. The form contains the following fields and controls:

- Project:** * s04_800_tidev_en (with a 'Browse...' button)
- Package:** * ZYTI_HANA (with a 'Browse...' button)
- ☐ Add to favorite packages
- Name:** * YGK_CDS_TEST
- Description:** * TESTING CDS VIEW
- Original Language:** EN
- Referenced Object:** (empty field with a 'Browse...' button)

At the bottom, there is a row of buttons: '< Back' (disabled), 'Next >' (active/highlighted), 'Finish' (disabled), and 'Cancel' (disabled). A help icon (?) is on the far left.

4. Choose Transport request and click on Next.



The 'New Data Definition' dialog box is shown at the 'Selection of Transport Request' step. It has a title bar with a question mark icon, a close button, and a maximize button. The main title is 'Selection of Transport Request' with a subtitle 'For YGK_CDS_TEST (DDL), the selected transport request will be used'. There is a blue truck icon in the top right corner. The form contains the following elements:

- ☒ Choose from requests in which I am involved (with a 'Configure Columns' button)
- type filter text (input field)
- Table of transport requests:

Transport Request	Owner	Target	Description	Last
S04K900041	TIDEV		SCU Test	Oct
S04K900039	TIDEV		HANA TRAINING	Oct
S04K900037	TIDEV		TEST REQUEST	Sep
S04K900031	TIDEV		Eclips Programs	Sep
S04K900027	TIDEV		File Path Create	Sep
S04K900025	TIDEV		TEST REQUEST	Sep

Below the table, there are two radio buttons:

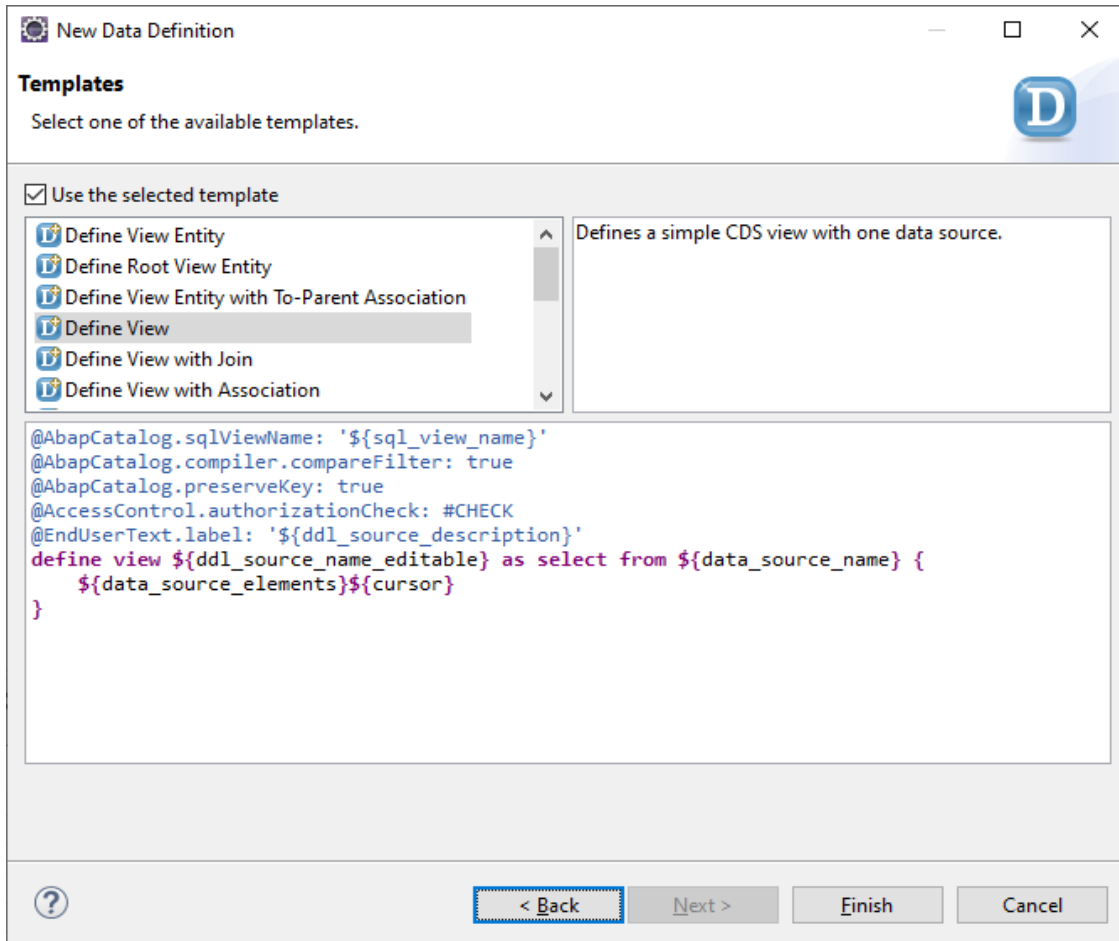
- ☐ Create a new request
- ☐ Enter a request number

For 'Create a new request', there is a 'Request Description: *' input field.

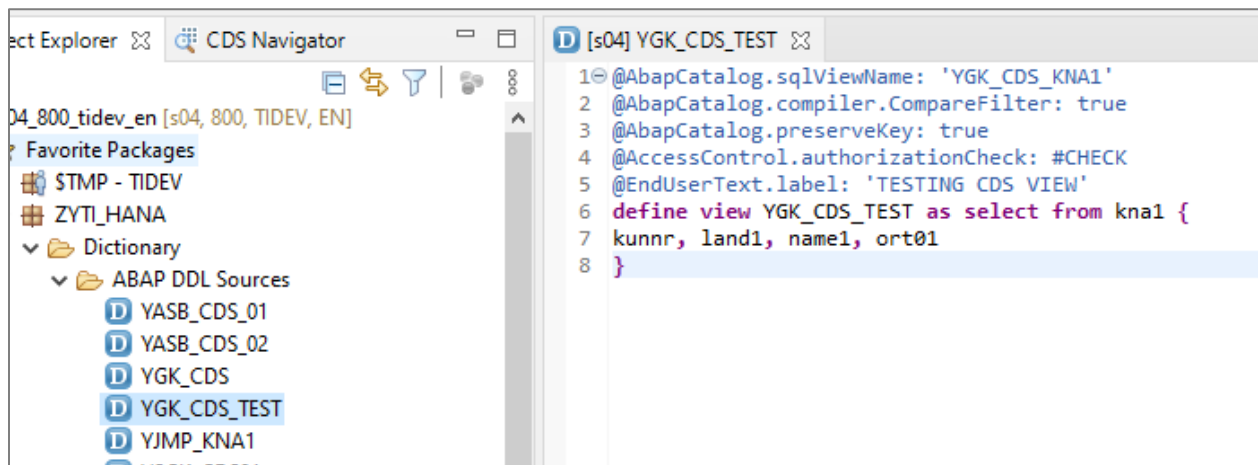
For 'Enter a request number', there is a 'Request Number:' input field and a 'Browse...' button.

At the bottom, there is a row of buttons: '< Back' (disabled), 'Next >' (disabled), 'Finish' (active/highlighted), and 'Cancel' (disabled). A help icon (?) is on the far left.

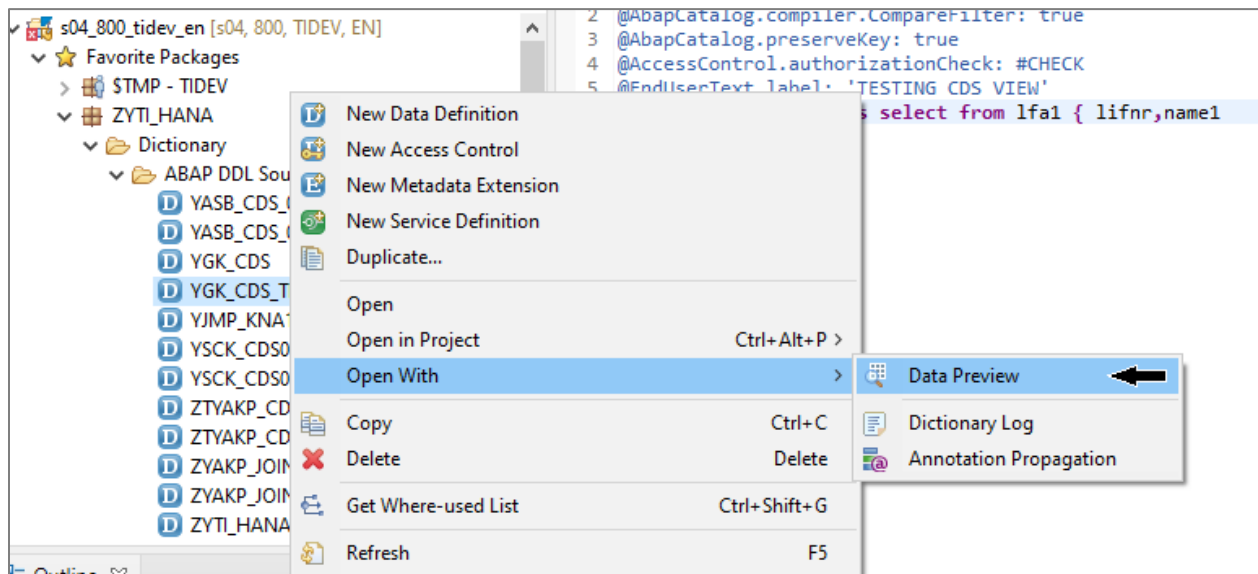
5. Here you can see various templates for the CDS views. You can choose the template as per the requirement and click on Finish button. Here we are choosing 'Define View' template from the template list.



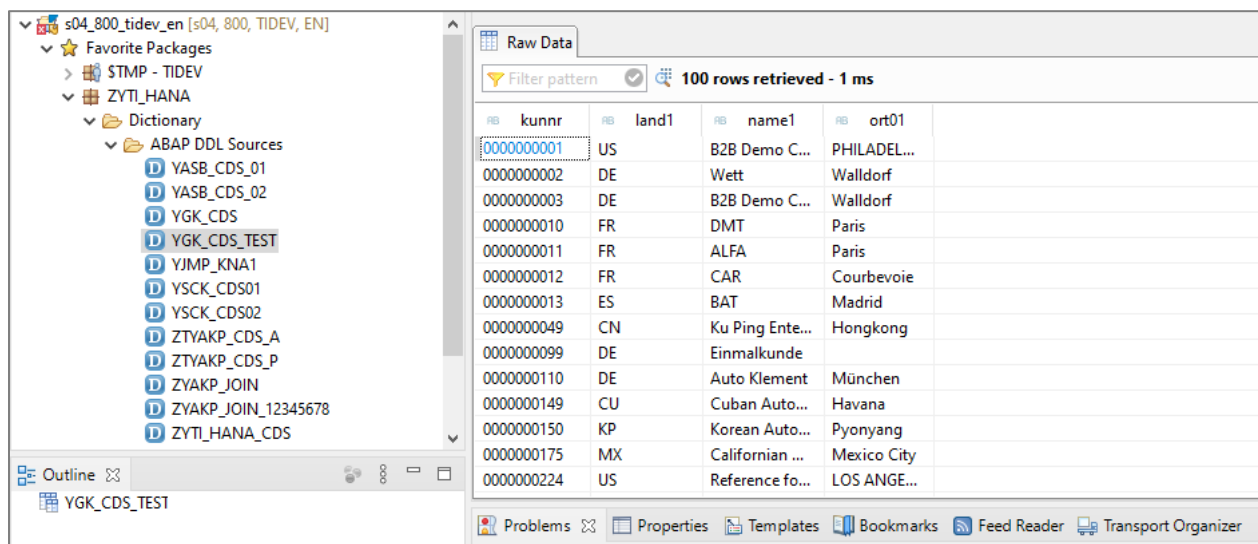
6. Write code for the CDS view as per the requirement.



7. You can get the preview of the created CDS view by right clicking on CDS view -> Open With -> Data Preview.

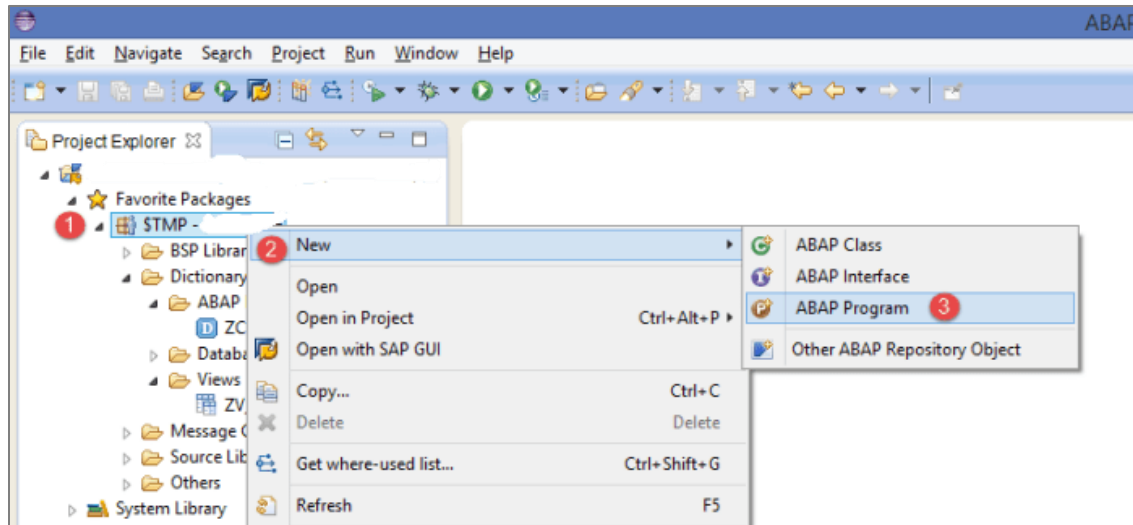


Data will be displayed in the Raw Data tab like below.

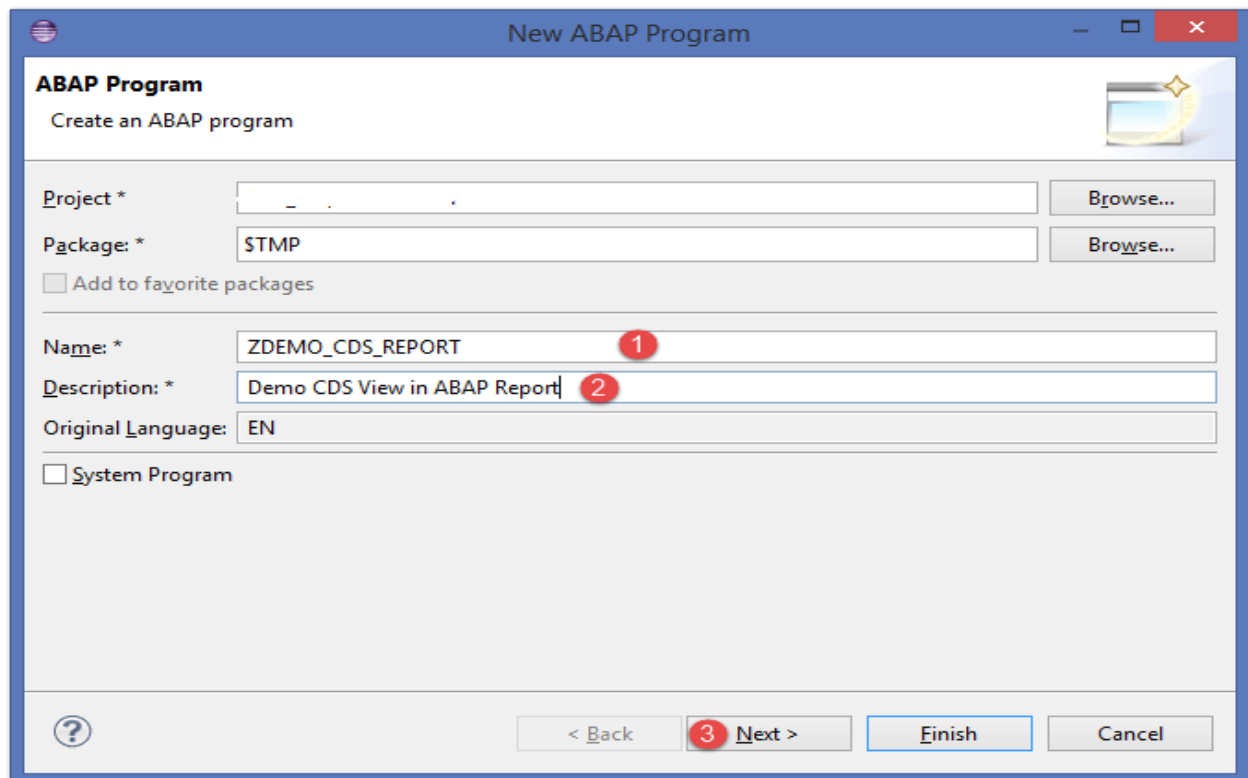


2. Calling CDS Views in ABAP report

1. Create an ABAP program/report.



2. In the **New ABAP Program** window, enter Name and Description and hit **Next**.



3. In the Selection of Transport Request window, choose the transport request. As we are saving the *program in \$TMP package* in our case no transport request is required. Click on Finish.

The screenshot shows the 'New ABAP Program' dialog box with the 'Selection of Transport Request' tab selected. The dialog has a title bar with standard window controls. Below the title bar, there is a message: 'No change recording enabled for package \$TMP'. The main area contains three radio buttons: 'Choose from requests in which I am involved' (selected, marked with a red circle '1'), 'Create a new request', and 'Enter a request number'. Below the 'Create a new request' option is a text field for 'Request description:'. Below the 'Enter a request number' option is a text field for 'Request number:' and a 'Browse...' button. At the bottom of the dialog, there are four buttons: a help button (question mark), '< Back', 'Next >', and 'Finish' (marked with a red circle '2').

To call an ABAP CDS view you can use Open SQL statements to get the data. Sample ABAP code snippet to call a CDS will look like below.

Syntax: `SELECT * FROM <cds_view_name>.`

```
*****
* OPEN SQL statement to access the CDS View *
*****
SELECT * FROM YGK_CDS_KNA1 INTO TABLE @DATA(It_data).
```

We can use the CDS View to create an ABAP ALV report using following code.

```

*****
*   CDS View in ABAP ALV Report   *
*****
DATA: lo_alv TYPE REF TO if_salv_gui_table_ida.

cl_salv_gui_table_ida=>create(
  EXPORTING
    iv_table_name      = 'YGK_CDS_KNA1' "Name of CDS View name
  RECEIVING
    ro_alv_gui_table_ida = lo_alv
).
* CATCH cx_salv_db_connection.  "
* CATCH cx_salv_db_table_not_supported.  "
* CATCH cx_salv_ida_contract_violation.  "

* Display ALV
lo_alv->FULLSCREEN( )->display( ).

```

5. We have used CDS view along with ALV with IDA class CL_SALV_GUI_TABLE_IDA as a data source to the ALV. Save and activate the ABAP program.

3. JOINS in ABAP CDS Views on HANA

1. INNER JOIN or just JOIN

Selects all entries which exists in both tables which meet ON condition.

Below is the sample ABAP CDS views in which INNER JOIN is used among 3 tables to get the so_id, so_item_pos, product_id, amount and currency code.

```

@AbapCatalog.sqlViewName: 'YGK_CDS_JOIN1'
@ClientDependent: true
@AbapCatalog.compiler.CompareFilter: true
@EndUserText.label: 'Demo'
define view YGK_CDS_JOIN as
select from snwd_so
  join snwd_so_i on snwd_so.node_key = snwd_so_i.parent_key

```

```

    inner join snwd_pd on snwd_so_i.product_guid = snwd_pd.node_key
{
  snwd_so.so_id,
  snwd_so_i.so_item_pos,
  snwd_pd.product_id,
  snwd_so_i.net_amount,
  snwd_so_i.currency_code
}

```

2. LEFT OUTER JOIN

Selects all records from the left table matching with the right table which meet ON condition. If no record found in second table those record fields are null.

```

@AbapCatalog.sqlViewName: 'YGK_CDS_LJOIN1'
@AbapCatalog.compiler.CompareFilter: true
@AccessControl.authorizationCheck: #CHECK
@endUserText.label: 'Joins in ABAP CDS Views'
define view YGK_CDS_LJOIN as select from snwd_so
  left outer join snwd_so_i on snwd_so.node_key = snwd_so_i.parent_key
{
  snwd_so.so_id,
  snwd_so.billing_status,
  snwd_so.delivery_status,
  snwd_so.op_id,
  snwd_so_i.so_item_pos,
  snwd_so_i.net_amount,
  snwd_so_i.currency_code
}

```

3. RIGHT OUTER JOIN

Selects all records from the right table matching with the left table which meet ON condition. If no record found in left table those record fields are null.

```

@AbapCatalog.sqlViewName: 'YGK_RJOIN1'
@AbapCatalog.compiler.CompareFilter: true
@AccessControl.authorizationCheck: #CHECK
@endUserText.label: 'Joins in ABAP CDS Views'

```

```

define view YGK_RJOIN as select from snwd_so
  right outer join snwd_so_i
  on snwd_so.node_key = snwd_so_i.parent_key
{
  snwd_so.so_id,
  snwd_so.billing_status,
  snwd_so.delivery_status,
  snwd_so.op_id,
  snwd_so_i.so_item_pos,
  snwd_so_i.net_amount,
  snwd_so_i.currency_code
}

```

4.CDS Views with Parameters

We can provide the parameters to the ABAP CDS Views by adding the syntax **WITH**

PARAMETERS p1, p2... Data Dictionary ABAP data types can be used while defining the parameters to the ABAP CDS Views.

```

@AbapCatalog.sqlViewName: 'YGK_CDS_PARAMS'
@AbapCatalog.compiler.compareFilter: true
@AccessControl.authorizationCheck: #CHECK
@endUserText.label: 'CDS View with Parameters'

```

```

define view ygk_Cds_With_Params
  with parameters p_billing_status :SNWD_SO_CF_STATUS_CODE,
    p_delivery_status :SNWD_SO_OR_STATUS_CODE
  as select from snwd_so
  join snwd_so_i on snwd_so.node_key = snwd_so_i.parent_key{
    snwd_so.so_id as orderno,
    snwd_so_i.so_item_pos as itemno,
    snwd_so_i.currency_code as currency,
    snwd_so_i.gross_amount as grossamount,

```

```
snwd_so_i.net_amount    as netamount,  
snwd_so_i.tax_amount    as taxamount
```

```
} where snwd_so.billing_status = :p_billing_status and
```

```
snwd_so.delivery_status = $parameters.p_delivery_status;
```

Now we will see how to call the ABAP CDS views with input parameters in an ABAP program

using Open SQL statement. Create an ABAP program in eclipse ADT or SE38 transaction. Below

is the code snippet to call the ABAP CDS Views with input parameters.

```
*-----*  
* Use Open SQL statement to get the data from the ABAP CDS views *  
* with Input Parameters                                           *  
*-----*  
  
SELECT orderno,  
       itemno,  
       currency,  
       grossamount,  
       netamount,  
       bill_status  
FROM ygk_cds_params( p_billing_status = 'P',  
                    p_delivery_status = 'D' )  
INTO TABLE @DATA(lt_result).  
  
* Display  
cl_demo_output=>display( lt_result ).
```

Input parameters can be used in different places in the SELECT of the ABAP CDS view. Below are ABAP CDS views examples in different forms.

1. Parameters used in SELECT list

In the below CDS view parameter “p_billing_status” is used in the SELECT list

```
@AbapCatalog.sqlViewName: 'ygk_cds_params'
@AbapCatalog.compiler.compareFilter: true
@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'cds view with parameters'

define view ygk_cds_with_params
  with parameters p_billing_status :snwd_so_cf_status_code,
                 p_delivery_status :snwd_so_or_status_code
  as select from snwd_so
  join snwd_so_i on snwd_so.node_key = snwd_so_i.parent_key{
    snwd_so.so_id      as orderno,
    snwd_so_i.so_item_pos as itemno,
    snwd_so_i.currency_code as currency,
    snwd_so_i.gross_amount as grossamount,
    snwd_so_i.net_amount as netamount,
    snwd_so_i.tax_amount as taxamount,

    //Parameters used in SELECT list
    :p_billing_status as billing_status

  } where snwd_so.billing_status = :p_billing_status and
         snwd_so.delivery_status = $parameters.p_delivery_status;
```

2. Parameters used in Arithmetic Expressions

In below CDS view code parameter “p_discount_percentage” is used in the arithmetic expressions to calculate the discount amount.

```
@AbapCatalog.sqlViewName: 'ygk_cds_params'
@AbapCatalog.compiler.compareFilter: true
@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'cds view with parameters'
```

```

define view ygk_cds_with_params
  with parameters p_billing_status    :snwd_so_cf_status_code,
                  p_discount_percentage :int4
  as select from snwd_so
  join snwd_so_i on snwd_so.node_key = snwd_so_i.parent_key{
    snwd_so.so_id      as orderno,
    snwd_so_i.so_item_pos as itemno,
    snwd_so_i.currency_code as currency,
    snwd_so_i.gross_amount as grossamount,
    snwd_so_i.net_amount  as netamount,

    // Parameter used in Arithmetic expressions
    (snwd_so_i.net_amount * :p_discount_percentage) as discount_amt

  } where snwd_so.billing_status = :p_billing_status;

```

3. Parameters used in CASE statement

In the below CDS view code the input parameter “p_billing_status” is used in CASE distinction.

```

@AbapCatalog.sqlViewName: 'ygk_cds_params'
@AbapCatalog.compiler.CompareFilter: true
@AccessControl.authorizationCheck: #CHECK
@endUserText.label: 'cds view with parameters'

```

```

define view ygk_cds_with_params
  with parameters p_billing_status    :snwd_so_cf_status_code
  as select from snwd_so
  join snwd_so_i on snwd_so.node_key = snwd_so_i.parent_key{
    snwd_so.so_id      as orderno,
    snwd_so_i.so_item_pos as itemno,
    snwd_so_i.currency_code as currency,
    snwd_so_i.gross_amount as grossamount,
    snwd_so_i.net_amount  as netamount,

    // Parameter used in CASE distinction
    case :p_billing_status
      when 'P' then 'Paid'
      else 'UnPaid'
    end          as bill_status
  }

```

```
} where snwd_so.billing_status = :p_billing_status;
```

5. Aggregate Expressions in ABAP CDS Views

Below is the list of aggregation expressions used in ABAP CDS views.

1. SUM

```
@AbapCatalog.sqlViewName: 'YGK_CDS_AGGR'  
@AbapCatalog.compiler.compareFilter: true  
@AccessControl.authorizationCheck: #CHECK  
@EndUserText.label: 'Aggregations'
```

```
define view ygk_Cds_Agg_sum as select from snwd_stock  
  join snwd_pd on snwd_stock.product_guid = snwd_pd.node_key  
  {  
    key snwd_pd.product_id,  
    snwd_pd.category,  
    // Aggregate function "SUM"  
    sum(snwd_stock.quantity) as total_stock  
  }  
  group by snwd_pd.category,  
    snwd_pd.product_id
```

In the above example ABAP CDS view selects the total stock of the product by using the aggregate function **SUM** and **GROUP BY** product and category.

2. MAX

```
@AbapCatalog.sqlViewName: 'YGK_CDS_AGGR'  
@AbapCatalog.compiler.compareFilter: true  
@AccessControl.authorizationCheck: #CHECK  
@EndUserText.label: 'Aggregations'  
define view ygk_Cds_Agg_max as select from snwd_so  
  join snwd_bpa on snwd_so.buyer_guid = snwd_bpa.node_key
```

```

{
  key snwd_bpa.bp_id,
    snwd_bpa.company_name,
  // Aggregate function "MAX"
  max(snwd_so.gross_amount) as max_sales_amt
} group by snwd_bpa.bp_id,
  snwd_bpa.company_name

```

In the above example ABAP CDS view selects the maximum sales amount generated by the customer by using the aggregate function **MAX** and **GROUP BY** business partner.

3. MIN

```

@AbapCatalog.sqlViewName: 'YGK_CDS_AGGR'
@AbapCatalog.compiler.CompareFilter: true
@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'Aggregations'
define view ygk_Cds_Agg_min as select from snwd_so
  join snwd_bpa on snwd_so.buyer_guid = snwd_bpa.node_key
  {
    key snwd_bpa.bp_id,
      snwd_bpa.company_name,
    // Aggregate function "MIN"
    min(snwd_so.gross_amount) as min_sales_amt
  } group by snwd_bpa.bp_id,
    snwd_bpa.company_name

```

In the above example ABAP CDS view selects the minimum sales amount generated by the customer by using the aggregate function **MIN** and **GROUP BY** business partner.

4. COUNT(*)

```

@AbapCatalog.sqlViewName: 'YGK_CDS_AGGR2'
@AbapCatalog.compiler.compareFilter: true
@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'Aggregations'
define view YGK_CDS_AGG as select from snwd_so
  join snwd_bpa on snwd_so.buyer_guid = snwd_bpa.node_key
  {
    key snwd_bpa.bp_id,

```

```

        snwd_bpa.company_name,
    // Aggregate expression COUNT( * )
    count(*) as min_sales_amt
} group by snwd_bpa.bp_id,
           snwd_bpa.company_name

```

In the above example ABAP CDS view selects the total number of sales order created against the business partner by using the aggregate function **COUNT(*)** and **GROUP BY** business partner.

5. COUNT(DISTINCT)

```

@AbapCatalog.sqlViewName: 'YGK_CDS_AGGR_4'
@AbapCatalog.compiler.CompareFilter: true
@AccessControl.authorizationCheck: #CHECK
@endUserText.label: 'Aggregate Expressions'
define view ygk_cds_Agg_C as select from snwd_so_i
    join snwd_pd on snwd_so_i.product_guid = snwd_pd.node_key
    {
        key snwd_pd.product_id,
        // Aggregate Expression - COUNT( DISTINCT )
        count( distinct snwd_so_i.node_key) as orders_count
    } group by snwd_pd.product_id

```

In the above example ABAP CDS view selects the total number of sales order created against the product by using the aggregate function **COUNT(DISTINCT)** and **GROUP BY** product id.

Points to be remembered

1. Every aggregate expressions used need an alternative element name defined using **AS**.
2. Aggregate expressions should require **GROUP BY** clause.
3. All non-aggregated fields used in CDS view should be specified in the **GROUP BY** clause.

6. String Functions in ABAP CDS Views

Below is the list of frequently used string functions in ABAP CDS views.

1.CONCAT(arg1, arg2)

CONCAT(arg1, agr2) string function can be used to concatenate two character strings.

```
@AbapCatalog.sqlViewName: 'YGK_CDS_STR_FUN'
@AbapCatalog.compiler.CompareFilter: true
@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'String Functions'
define view ygk_cds_Sql_Func as select from kna1 {

    // CONCATENATE name1 & name2
    CONCAT( kna1.name1, kna1.name2 ) as full_name
}
```

2. CONCAT_WITH_SPACE(arg1, arg2, spaces)

This string function is used to concatenate two character strings with space. The number of blanks between the arguments *arg1* and *arg2* is specified in *spaces*.

```
@AbapCatalog.sqlViewName: 'YGK_CDS_STR_FUN'
@AbapCatalog.compiler.compareFilter: true
@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'String Functions'
define view ygk_cds_Sql_Func as select from kna1 {

    // CONCATENATE name1 & name2 with 4 space
    CONCAT_WITH_SPACE( kna1.name1, kna1.name2, 4 ) as full_name
}
```

3 . SUBSTRING(arg, pos, len)

To get sub string of *arg* from the position *pos* in the lenght *len*.

```
@AbapCatalog.sqlViewName: 'YGK_CDS_STR_FUN'
@AbapCatalog.compiler.CompareFilter: true
```

```

@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'String Functions'
define view ygk_cds_Sql_Func as select from kna1 {

    // To get substring for a given string
    SUBSTRING( kna1.name1, 2, 10) as name
}

```

4 . LENGTH(arg)

It returns the no of characters in the string which is passed as a argument *arg*. It ignores trailing blanks.

```

@AbapCatalog.sqlViewName: 'YGK_CDS_STR_FUN'
@AbapCatalog.compiler.CompareFilter: true
@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'String Functions'
define view ygk_cds_Sql_Func as select from kna1 {

    // To get length for a given string
    LENGTH( kna1.name1 ) as name_length
}

```

5 LEFT(arg, len) & RIGHT(arg, len)

LEFT(arg, len) – It returns the left-side part of the string which is passed as argument *arg* of length *len*.

RIGHT(arg, len) – It returns the right-side part of the string which is passed as argument *arg* of length *len*.

Note: Trailing blanks are ignored

```

@AbapCatalog.sqlViewName: 'YGK_CDS_STR_FUN'
@AbapCatalog.compiler.CompareFilter: true
@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'String Functions'
define view ygk_cds_Sql_Func as select from kna1 {

    // To get length for a given string

```

```

LEFT( kna1.name1, 3) as name_left,
RIGH( kna1.name1, 3) as name_right,
kna1.name1
}

```

6 LTRIM(arg, char) & RTRIM(arg, char)

LTRIM(arg, char) – It removes the trailing blanks and leading character which matches the parameter *char*.

RTRIM(arg, char) – It removes the trailing blanks and trailing character which matches the parameter *char*.

```

@AbapCatalog.sqlViewName: 'YGK_CDS_STR_FUN'
@AbapCatalog.compiler.CompareFilter: true
@AccessControl.authorizationCheck: #CHECK
@endUserText.label: 'String Functions'
define view ygk_cds_Sql_Func as select from kna1 {

    // Removes the trailing blanks and character
    LTRIM( kna1.name1, 'L') as name_lt,
    RTRIM( kna1.name1, 'T') as name_rt
}

```

7.CDS View Extensions

A CDS view is extended by creating a new DDL source with DDL statement **EXTEND VIEW**.

As we know for every DDL source we have 2 corresponding dictionary objects created:

- The CDS View and
- SQL View

So when you extend a CDS view, there will be two corresponding objects created:

- The CDS view extension and
- Append View

In DDL source code editor, enter the necessary information for names of the append view, the original CDS view and for the actual view extension like below.

